

Game Design Document — Red Flag

Simulador de Gestión de Riesgo Bancario

FBD Studio · Ingeniería de Videojuegos: Fundamentos y Aplicaciones Interactivas · USACH — Departamento de Ingeniería Informática LAB 6 CIERRE — Entrega final (Julio 2026)

Sección 1 — Identificación (*prioridad alta*)

Campo	Detalle
Nombre del proyecto	Red Flag — Simulador de Gestión de Riesgo Bancario
Equipo / estudio	FBD Studio
Integrantes	Diego Altamirano · Felipe Cifuentes · Byron Obregón
Motor	Unity 6.4 (6000.4.1f1), C#
Plataforma	Web (WebGL) como objetivo de distribución; ejecutable también desde el Editor de Unity y build de Windows
Género	Serious game de simulación de gestión / verificación documental bajo presión de tiempo
Audiencia	18–65 años, perfil casual, en contexto de escritorio (oficina / home office); colaboradores nuevos y equipos operativos de instituciones financieras
Repositorio	https://github.com/dialtamiranoh/usach-ingevi-redflag

Red Flag es un serious game 3D que gamifica el entrenamiento en gestión de riesgo bancario bajo el marco regulatorio chileno. El jugador asume el rol de un analista de cumplimiento que atiende clientes, revisa expedientes documentales y decide **Aprobar / Escalar / Rechazar** cada caso, apoyándose en normativa real (Ley 19.913, Ley 20.393, Ley 21.521 y Circular UAF N°62).

Sección 2 — Visión y propuesta de valor (*prioridad alta*)

Elevator pitch (3 frases).

1. Red Flag pone al jugador en la silla de un analista de cumplimiento bancario que, contra el reloj, debe leer expedientes, interrogar clientes y detectar las "red flags" que delatan lavado de activos o fraude.
2. Cada decisión —aprobar, escalar o rechazar— es un ejercicio de compliance real anclado en la normativa chilena vigente, con consecuencias inmediatas en el puntaje y en la continuidad de la jornada.
3. Es un entrenamiento profesional disfrazado de juego de tensión al estilo *Papers, Please*, pensado para transformar una capacitación obligatoria y repetitiva en una experiencia interactiva y memorable.

Diferenciador (una oración). No existe hoy ningún juego o simulador que gamifique el entrenamiento en gestión de riesgo bancario **bajo el marco regulatorio chileno** (UAF, Circular N°62), lo que posiciona a Red Flag como una herramienta de e-learning con aplicabilidad profesional directa.

Referentes. *Papers, Please* (verificación documental y decisión bajo presión con causa válida) y *That's not my Neighbor* (checklist de verificación ítem por ítem antes de decidir).

Sección 3 — Diseño del juego (*prioridad alta*)

3.1 Tabla MDA

Capa	Contenido del proyecto
Mecánicas	(1) Evaluación documental y decisión KYC/AML (Aprobar / Escalar / Rechazar). (2) Interrogatorio al cliente por clic en los campos del expediente o en el modelo 3D. (3) Gestión de objetos del escritorio por drag & drop (guardar objetos de seguridad, ignorar sobornos). (4) Puntuación con multiplicador por racha de aciertos. (5) Agente NPC (Gerente) que interrumpe con preguntas de normativa contra reloj.
Dinámicas	(1) Gestión de riesgo bajo presión de tiempo: el cronómetro por caso obliga a priorizar. (2) Lectura e interpretación de señales (red flags): contrastar documentos, transacciones y respuestas para detectar incoherencias. (3) Búsqueda de la racha: el multiplicador incentiva mantener decisiones correctas consecutivas. (4) Gestión de la atención: el segundo temporizador del Gerente crea una decisión táctica permanente (responder ya vs. terminar de revisar).
Estéticas	(1) Tensión y responsabilidad profesional (el error tiene consecuencias). (2) Desafío y descubrimiento al destapar el caso. (3) Sensación de progreso y dominio mediante la racha y el puntaje creciente.

3.2 Mecánica principal

El núcleo es un **bucle de juicio profesional**: se presenta un cliente y su expediente; el jugador alterna entre tres vistas de cámara —**Cliente** (frente al modelo 3D), **Monitor** (tecla **Q**, documentos KYC/AML) y **Notepad** (tecla **E**, expediente; **Esc** o la misma tecla para volver)— para reunir evidencia, interroga al cliente haciendo clic en los campos, contrasta discrepancias (resaltadas con la clase visual `field-alerta`) y emite una decisión **Aprobar / Escalar / Rechazar**. La cámara se mueve con interpolación suave dependiente de `Time.deltaTime` para ser independiente del framerate. En paralelo, aparecen **objetos sospechosos** sobre el escritorio (pendrives, celulares, sobres) que deben arrastrarse al cajón; los **sobornos** no deben tocarse.

El **sistema de jugabilidad** implementado es *Puntuación con multiplicador/combo*: decisiones correctas consecutivas elevan el multiplicador ($\times 1 \rightarrow \times 1.5 \rightarrow \times 2 \dots$ tope $\times 3$) y un error lo rompe. La conexión con el HUD usa el **patrón Observer** (eventos C# `OnPuntajeChanged`, `OnMultiplicadorChanged`, `OnTurnoChanged`), de modo que la interfaz reacciona a eventos y no consulta el estado en cada frame.

Condiciones de fin de jornada:

Desenlace	Cuándo se activa
Victoria	Completar la jornada con puntaje $\geq 500 \rightarrow$ «Jornada exitosa».
Derrota por puntaje	El puntaje cae a 0 tras un error en turnos avanzados $\rightarrow$ «Demasiados errores de compliance».
Derrota por tiempo	El cronómetro del caso llega a 0 $\rightarrow$ «Tiempo agotado».

3.3 Layout del nivel

La acción transcurre en un único escenario —el escritorio del analista dentro de una oficina low-poly— navegable con las tres vistas de cámara. **Punto de inicio**: pantalla de introducción narrativa (SceneTutorial). **Punto de final**:

pantalla de resultado de la jornada. Los objetos sospechosos reaparecen en puntos de spawn predefinidos sobre el escritorio. La **curva de dificultad** escala por densidad: `ObjetosManager` reduce el intervalo de aparición al avanzar la jornada ($\max(10, 25 - \text{turno} \times 3)$) s para objetos de seguridad; $\max(15, 30 - \text{turno} \times 2)$ s para sobornos, y los casos finales concentran el clímax (PEP, fraccionamiento/smurfing, múltiples discrepancias).

La progresión de rol añade profundidad espacial: **Nivel 1 — Analista KYC**, **Nivel 2 — Analista KYC/AML** (se habilita un segundo escritorio de Back-Office, consultable con la tecla **W**, y formularios ROS), y **Nivel 3 — Supervisor Corporativo** (evalúa casos escalados y fundamenta rechazos con un Reporte de Operación Sospechosa).

Nota: el layout anotado con punto de inicio/final, spawns y zona de dificultad está en la entrega LAB 4; puede incluirse como figura fotografiada/exportada en la versión final del GDD.

3.4 Agente autónomo — Gerente de Sucursal (3 estados)

El NPC autónomo es el **Gerente de Sucursal**: recorre la oficina por cuenta propia y, cada cierto tiempo, se acerca al escritorio activo para interrogar al analista con una pregunta de normativa (Ley 19.913 / Ley 20.393 / Circular UAF N°62) que debe responderse en ≤ 15 s. Se gobierna con una **Máquina de Estados Finitos (FSM)** de tres estados con patrón OnEnter/Update/OnExit —elegida sobre un Árbol de Comportamiento por no requerir prioridades dinámicas ni composición de tareas—:

Estado	Qué hace (OnEnter / Update / OnExit)	Transición
Patrol	Reanuda el agente, anima <code>IsWalking</code> , fija waypoint/punto aleatorio y reinicia el timer; en Update rota entre waypoints y descuenta el timer de visita.	$\text{timer} \leq 0 \rightarrow \text{Approach}$
Approach	<code>SetDestination()</code> al escritorio activo según nivel y rol (vía <code>RoleManager</code>); en Update mide la distancia horizontal al escritorio; OnExit detiene y rota al agente hacia el escritorio.	$\text{distancia} \leq \text{rangoInteraccion} \rightarrow \text{Interact}$
Interact	Anima <code>IsInteracting</code> y despliega el popup de pregunta con timer de 15 s (delegado en <code>GerenteInteractUI</code>); OnExit oculta el popup.	respuesta correcta $\rightarrow$ daño al gerente y vuelve a Patrol; incorrecta/timeout $\rightarrow$ -100 pts + reset de racha y vuelve a Patrol

- **Navegación:** NavMesh nativo de Unity (`NavMeshAgent`, radius 0.3, height 1.8, speed 2.5, obstacle avoidance High Quality); destinos validados con `NavMesh.SamplePosition` y auto-corrección si una colisión lo saca de la malla.
- **Detección:** por rango configurable (`rangoInteraccion`: 0.5 m en escena, 1.5 m por defecto), midiendo distancia al escritorio activo.
- **Integración bidireccional:** inflige "daño" restando puntaje vía métodos públicos de `UIManager` (nunca variables directas), disparando la cadena de eventos del LAB 4; recibe daño por respuestas correctas (`GerenteNPC_Health`, 3 HP, barra flotante World-Space con Billboard), y al morir emite un evento observable que actualiza el contador y muestra un toast "Gerente retirado". Las preguntas se cargan desde `npcPreguntas.json` (contenido dirigido por datos).

Sección 4 — Arquitectura técnica (prioridad media)

Tecnología de interfaz: UI Toolkit (UXML + USS), sin Gizmos/Debug para la UI de juego.

Componentes más importantes:

- `UIManager.cs` — Orquestador central: gestiona vistas, turnos, timer, puntaje y multiplicador, y expone los **eventos Observer** a los que se suscribe el HUD. Es el punto de integración de todos los sistemas (decisiones, feedback visual, condición de fin, contador de gerentes).
- `CaseManager.cs` — Carga los casos desde `casos.json` y valida la decisión correcta contra las discrepancias del expediente.
- `GerenteNPC_FSM.cs` + `GerenteNPC_Health.cs` + `GerenteInteractUI.cs` — El agente autónomo (FSM, salud, UI de pregunta) del LAB 5.
- `CameraController.cs` — Sistema de 3 vistas con transición por delta time. `RoleManager.cs` — progresión de niveles/roles y evento `OnNivelCambiado`.

Decisión técnica relevante (con justificación): conectar el HUD al estado del juego mediante el **patrón Observer** (eventos C#) en lugar de polling en `Update`. Justificación: desacopla la lógica de juego de la presentación, permite que múltiples elementos del HUD reaccionen al mismo evento (puntaje, multiplicador, barra de progreso) y evita recalcular estado por frame. La misma infraestructura de eventos se reutilizó en el LAB 5 (evento de "gerente retirado"), demostrando su extensibilidad.

Deuda técnica honesta (bugs conocidos):

- Inconsistencia de conteo de turnos:** el diagrama de flujo y algunas pantallas muestran "TURNOS n / 12" mientras el diseño documentado habla de 5 turnos por jornada (`turnoTotal = 12` en `UIManager.cs`); conviene unificar diseño, HUD y flujo en una sola fuente de verdad.
- Logs de depuración:** `GerenteNPC_FSM.cs` conserva numerosos `Debug.Log` de diagnóstico de navegación que deberían silenciarse en la build final.
- NavMesh:** casos borde de waypoints mal ubicados / empujes fuera de la malla ya mitigados con fallback aleatorio y auto-warp, pero dependientes de un horneado correcto de la escena.
- ~~(Corregido en LAB 6, Ajuste 1) Textos de control contradictorios entre tutorial, README y comentarios del código, y tecla Esc documentada pero no implementada.~~ Se unificó todo al mapeo real (**Q=Monitor**, **E=Notepad**, ambos toggle) y se implementó **Esc** para volver a la vista del cliente.

Sección 5 — Proceso de desarrollo (prioridad media)

Tabla de hitos:

Hito	Entrega	Estado
LAB 1 — Propuesta de proyecto	12-Abr-2026	Completado
LAB 2 — Prototipo de escena base	Semana 5	Completado
LAB 3 — Mecánica principal (física + input)	Semana 7-8	Completado
LAB 4 — Jugabilidad y diseño de niveles (HUD, Observer, fin de jornada)	27-Jun-2026	Completado
LAB 5 — IA y NPC (FSM 3 estados + NavMesh)	05-Jul-2026	Completado
LAB 6 CIERRE — Prototipo funcional + GDD + playtesting + presentación	24-Jul-2026	En cierre

Resumen de la sesión de playtesting (Bloque B): sesión del 21-07-2026 (~15 min) con una participante externa (31 años, experiencia nula en videojuegos, con conocimiento del dominio bancario). Hallazgos principales: confusión con los controles (Q/E, cámara, arrastre de objetos) que le costó su primera jornada, y frustración con la frecuencia e interrupciones del Gerente NPC (visitas cada 15 s, tiempo del caso consumido durante la pregunta, opciones KYC

desbordando el cuadro). Lo más disfrutado: la sensación de logro/aprendizaje y la música. De ahí derivaron los **2 ajustes implementados**: (1) rediseño de la lámina de controles del tutorial + leyenda persistente en el HUD; (2) pausa del timer del caso durante la pregunta del Gerente + espaciado de visitas (45/30/20 s) + fin de interrupciones tras jornada terminada + opciones en una columna con ajuste de texto. Detalle completo en el Documento de cierre.

Sección 6 — Dimensión aplicada (*prioridad media — integra el Bloque A*)

- **Clasificación**: Serious game (objetivo no lúdico primario: entrenamiento en gestión de riesgo bancario) con fuerte componente de **gamificación de un proceso** (KYC/AML/Compliance convertidos en mecánicas jugables). El entretenimiento es el medio, no el fin.
 - **Necesidad / valor**: útil para instituciones financieras chilenas y sus analistas (nuevos y operativos); transforma una capacitación obligatoria y repetitiva en práctica activa de detección de red flags y toma de decisiones bajo normativa real. Contexto: entre 2007 y 2023 la UAF reporta 323 sentencias condenatorias y decomisos por US\$58,1 millones por lavado de activos.
 - **Objetivo de impacto**: que quien juega sea capaz de **reconocer una discrepancia documental o transaccional (red flag) y decidir la acción de compliance correcta —aprobar, escalar o rechazar— justificándola con la normativa aplicable**.
-

Sección 7 — Resultados y conclusiones (*prioridad alta*)

Estado final del prototipo.

- *Funciona*: flujo completo Inicio → Tutorial → Jornada (revisión de casos, decisiones, drag & drop de objetos) → Gerente NPC con sus 3 estados → pantalla de resultados; HUD reactivo con multiplicador y condiciones de victoria/derrota; progresión de roles KYC → AML → Supervisor.
- *Con deuda conocida*: conteo de turnos inconsistente en HUD/flujo, logs de depuración del NPC y dependencia de un horneado correcto de NavMesh (ver Sección 4). Los textos de control inconsistentes y el ritmo del Gerente fueron corregidos en los ajustes del LAB 6 tras el playtesting.

Tres aprendizajes del equipo.

1. Traducir normativa compleja (UAF, AML/KYC) en mecánicas jugables claras exige recortar y priorizar: el valor formativo vive en la decisión, no en el texto.
2. Desacoplar lógica y presentación con el patrón Observer pagó dividendos: el mismo diseño de eventos absorbió sistemas nuevos (NPC) sin reescribir el HUD.
3. La IA de un NPC "simple" (FSM + NavMesh) es en la práctica un ejercicio de casos borde: la mayor parte del esfuerzo fue mitigar comportamientos no deseados (rutas rotas, empujes fuera de malla, doble respuesta).

Tres cosas que harían diferente.

1. Definir el conteo de turnos y el layout del nivel una sola vez, en una fuente de verdad, para evitar inconsistencias entre diseño, HUD y flujo.
 2. Introducir playtesting externo antes (desde el LAB 3/4) en lugar de solo en el cierre.
 3. Encapsular la configuración de balance (timers, penalizaciones, umbrales) en un ScriptableObject para iterar sin tocar código.
-

Sección 8 — Referencias (*prioridad baja*)

- Ley N°19.913 (crea la UAF); Ley N°20.393; Ley N°21.521 (Fintech); Circular UAF N°62.
- Unidad de Análisis Financiero (UAF), Chile — estadísticas de lavado de activos.
- Adams, E. (2014). *Fundamentals of Game Design* (3rd ed.). New Riders.
- Robinson, L. et al. (2020). *Serious Games and Edutainment Applications*. Springer.

- Unity — Manual y Scripting API (UI Toolkit, NavMesh, Input System). <https://docs.unity3d.com/>

Nota sobre uso de IA: Claude (Anthropic) fue utilizado como asistente en la revisión del prototipo y en la redacción y estructuración de la documentación del proyecto.